

```
extern void lcd_puts(const char *s);
Display string without auto linefeed.
extern void lcd_puts_p(const char
*program_s);
Display string from program memory
without auto linefeed.
extern void lcd_showvalue(unsigned
char num);
Displays the decimal 3digit value
of num.
extern void lcd_command(uint8_t cmd);
Send LCD controller instruction
command.
extern void lcd_data(uint8_t data);
Send data byte to LCD controller.
```

Declarations and definitions for the user-defined functions are included in `lcd.c` and `lcd.h` driver files. We have included a small sample project file named 'moving_display.c' to explain the working of these functions. The program prints a moving text message on the LCD module. To do this, it reads the message from the flash memory of the MCU and scrolls it on the LCD panel continuously.

First of all, you need to include header files in the program as follows:

```
#include <avr/io.h>
#include <avr/pgmspace.h>
#include <util/delay.h>
#include "lcd.h"
```

Here, `pgmspace.h` is an AVR-specific header file to incorporate program space string utilities in the program. For details of different functions available in a particular header file, refer to the `avr-libc` 'Reference' manual (included in the `EFY-CD`). The `lcd.c` is the driver external C source file for the LCD module. Add the `lcd.c` and `lcd.h` source files along with the `moving_display.c` application source file under 'Source Files' directory in the AVR Studio platform.

After including the required header files, initialise your LCD module in the main function with the required display attributes using the `lcd_init` (`uint8_t dispAttr`) function. The available display attributes are: `LCD_DISP_OFF` (for LCD off), `LCD_DISP_ON` (display on, cursor off), `LCD_DISP_ON_CURSOR` (display on, cursor on)

and `LCD_DISP_ON_CURSOR_BLINK` (display on, cursor on flashing).

In our program (`moving_display.c`), we have used the `LCD_DISP_ON` attribute to initialise the display. After initialising the display, clear it using the `lcd_clrscr` (void) function. In order to point the cursor at a desired location in the display, use the `lcd_gotoxy` (`uint8_t x`, `uint8_t y`) function. Here 'x' and 'y' denote the coordinates. For example, if you need to point the cursor to fifth position in the first row of the LCD module, the corresponding 'x' and 'y' coordinates would be 0 and 4, respectively. The `lcd_putc` (`char c`) function is used to print a single character at the current cursor location. Similarly, the `lcd_puts` (`const char *s`) function prints a string pointed to by 's' pointer from the current cursor location.

After successfully burning the hex code into the microcontroller, the message "ELECTRONICS FOR YOU – Asia's Largest Selling Electronics Magazine" can be seen scrolling continuously on the LCD module.

Example 4: Analogue-to-digital converter module

The `adc_test.c` source code is provided to test the working of the analogue-to-digital converter (ADC) module in ATmega32 MCU. The code measures the analogue voltage applied at the analogue input pin (PA7) of ATmega32 and prints the voltage value on the LCD module, correct up to three decimal places.

To test the code, first close switch S6 in the board so that preset VRI is connected to the MCU. VRI feeds an analogue voltage between 0V and 4.96V to pin 33 (PA7) of the MCU.

The algorithm used in the source code is as follows: First, take 256 samples of the analogue voltage applied at input pin 33 (PA7) and convert them into corresponding digital ADC values using the inbuilt ADC module. Then take the average of these 256 values to obtain a more accurate value. The average ADC value obtained is used

to measure the input voltage as follows:

$$ADC = \frac{V_{IN} \times 1024}{V_{REF}}$$

The entire cycle is repeated with the help of the ADC interrupt routine. The ADC interrupt routine is executed each time the analogue-to-digital conversion completes. In order to enable the ADC interrupt routine, set 'ADC Interrupt Enable' bit in `ADCSRA` register and 'Global Interrupt Enable' bit in `SREG` register. The interrupt routine is defined in the source code using `ISR()` macro defined in the `interrupt.h` header file.

ADC operation. The ADC converts an analogue input voltage into a 10-bit digital value through successive approximation. The minimum value represents GND and the maximum value represents the voltage on `AREF` pin minus 1 LSB. Optionally, `AVCC` or an internal 2.56V reference voltage may be connected to `AREF` pin by writing to the `REFSN` bits in `ADMUX` register. Refer to the datasheet for further details and operations.

Example 5: Timer module

The `8bit_timer.c` source code is provided to test the timer/counter0 module. It lights up the LEDs connected to Port B of the MCU in a sequential pattern to and fro. A program (`Chaser_Effect.c`) to light up the LEDs in a sequential manner has already been discussed. The program uses 'while' and 'for' loop combinations to produce the sequential pattern. In the `8bit_timer.c` code, use the `timer0/counter0` module to do the same task.

Here the `timer0/counter0` module is made to operate in 'clear timer on compare' mode. Clock source for the counter is the MCU frequency prescaled by a factor of 64. The comparator match interrupt routine is called every time the counter reaches the count of 250. That is, with every 250th count, 1 ms elapses.

EFY note. The example source codes and reference manual of this article have been included in this month's `EFY-CD` and are also available on www.efymag.com website.

To be concluded next month...